

AI Engineering Internship – Complete Documentation

Sawaira Mumtaz

Posts Fusion / Pro Fusion AI

July – August 2026

Table of Contents

1. Executive Summary
 2. Learning Journey
 3. Technical Implementation
 4. Prompt Engineering
 5. API Integration
 6. Full Stack Development
 7. Sample Outputs
 8. Challenges & Solutions
 9. Skills Acquired
 10. Conclusion
-

1. Executive Summary

This internship focused on building AI-powered content generation tools using various LLM APIs (Ollama, Groq, OpenRouter) and full-stack development with React + Node.js. The journey progressed from basic Python scripting to complex streaming applications with real-time token generation.

Key Deliverables:

-  LinkedIn Post Generator (Python + Ollama/Groq)
 -  Streaming Content Studio (React + Node.js + OpenRouter)
 -  7 Weeks of Progressive Learning & Implementation
-

2. Learning Journey

Week 1: Environment Setup & Python Fundamentals

Focus: Python refresher, virtual environments, API basics

Key Learnings:

- Python functions, dictionaries, JSON handling
- Making API calls (Weather API, OpenRouter API)
- Virtual environment setup and package management
- Understanding API authentication and headers

Code Snippet:

python

```
def call_ollama(system_message, user_message):  
    payload = {  
        "model": "mistral",  
        "messages": [  
            {"role": "system", "content": system_message},  
            {"role": "user", "content": user_message}  
        ],  
        "stream": False  
    }  
    response = requests.post("http://localhost:11434/api/chat", json=payload)  
  
    return response.json()["message"]["content"]
```

📁 Files: Tuesday_task.py, Wednesday_task.py, Thursday_task.py

Week 2: Prompt Engineering & Content Generation

Focus: Zero-shot vs Few-shot, tone control, constraint validation

Key Learnings:

- Zero-shot prompts generate generic, robotic content
- Few-shot examples (using your own work) dramatically improve quality
- Tone control is structural, not just vocabulary
- Iterative prompting turns 0% success rate into 100%

Example System Prompt (Detailed):

text

ROLE: You are a recruiting industry expert who writes educational LinkedIn content.

AUDIENCE: Job seekers, early-career professionals.

TONE GUIDELINES:

- Warm but professional
- Use relatable scenarios
- Avoid jargon

CONTENT RULES:

- Teach ONE specific lesson
- Include 3-4 actionable points
- End with a question

STRUCTURE:

1. Hook: Relatable problem
2. Body: Break down lesson
3. Call-to-action: Ask for engagement

📁 Files: Few-shot prompting.py, generate_6_posts.py, constraint_test.py

Week 3: LinkedIn Post Generator with Self-Check

Focus: AI self-critique, quality scoring, automation

Key Learnings:

- Forcing AI to rate its own output creates a quality gate
- Self-check scores (1-10) filter out generic content
- Different models produce different quality levels

Self-Check Function:

python

```
def rate_post(text):
    prompt = f"""
    Rate this LinkedIn post from 1-10:
    1-3: Very robotic, generic placeholders
    4-6: Decent structure but stiff
    7-8: Sounds human, good specific tips
    9-10: Excellent, feels like a real expert

    Post: {text}

    Return: Score and reason.
    """

    return call_ollama("You are a strict content critic.", prompt)
```

Sample Output:

text

Score: 5/10

Reason: The post uses generic statements and placeholders.

Lacks specific actionable numbers or real-world examples.

 Files: `linkedin_generator.py`, `README2.md`

Week 4: Groq API Integration

Focus: Speed, free tier, command-line LinkedIn generator

Key Learnings:

- Groq is significantly faster than local Ollama (100-300ms vs 1-3s)
- Free tier handles 30 requests/min, 15K tokens/min
- Real-time validation (word count 150-300, 3-5 hashtags)

Code Example:

```
python
```

```
from groq import Groq

client = Groq(api_key="your-key")

completion = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[{"role": "user", "content": "Write a LinkedIn post..."}],
    temperature=0.7,
    max_tokens=1024
)
```

Output Example:

```
text
```

```
=====
🚀 GROQ LINKEDIN GENERATOR
📌 Topic: 'AI in Healthcare' | Niche: Tech Leader
=====

--- GENERATED CONTENT ---
AI in Healthcare: The $60B Opportunity Founders Can't Ignore
...
📊 VALIDATION:
• Word Count: 208 (150-300) ✅
• Hashtags: 5/5 ✅
```

• Valid:  PASSED

 Files: `generate.py`, `README4.md`, `README_2.md`, `README_3.md`

Week 5: OpenRouter + Full-Stack Streaming App

Focus: React frontend, Node.js backend, SSE streaming, multiple models

Key Learnings:

- OpenRouter provides access to 50+ models through one API
- Server-Sent Events (SSE) enable real-time token streaming
- `model: "openrouter/free"` auto-selects a working free model
- Using `:free` suffix ensures no-cost inference

Tech Stack:

Layer	Technology
Backend	Node.js + Express + TypeScript
AI	OpenRouter API (OpenAI-compatible)
Streaming	Server-Sent Events (SSE)
Frontend	React + Vite + Tailwind

Code Example:

typescript

```
app.post('/api/content/generate', async (req, res) => {
  const { topic = "AI Productivity", niche = "SaaS Founders", tone =
"authoritative" } = req.body;

  res.setHeader('Content-Type', 'text/event-stream');
  res.setHeader('Cache-Control', 'no-cache');

  const sendSSE = (event, data) => {
    res.write(`event: ${event}\ndata: ${JSON.stringify(data)}\n\n`);
  };

  sendSSE('start', { topic, niche, tone });

  const stream = await openrouter.chat.completions.create({
    model: "openrouter/free",
    messages: [{ role: 'user', content: prompt }],
    stream: true,
  });

  for await (const chunk of stream) {
    const content = chunk.choices[0]?.delta?.content || '';
    if (content) {
      sendSSE('token', { token: content });
    }
  }

  sendSSE('done', { source: 'openrouter' });
  res.end();

});
```

📁 Files: server.ts, package.json, vite.config.ts, README5.md

Week 6: React UI + Streaming Improvements

Focus: Enhanced UI, CORS handling, error management

Key Learnings:

- Serving frontend and backend from same server avoids CORS
- Inline HTML + React components for rapid prototyping
- Visual feedback during streaming (token-by-token display)

📁 Files: `server-simple.js`, `test.html`, `README6.md`

Week 7: Final Deployment & Documentation

Focus: Production-ready code, comprehensive documentation

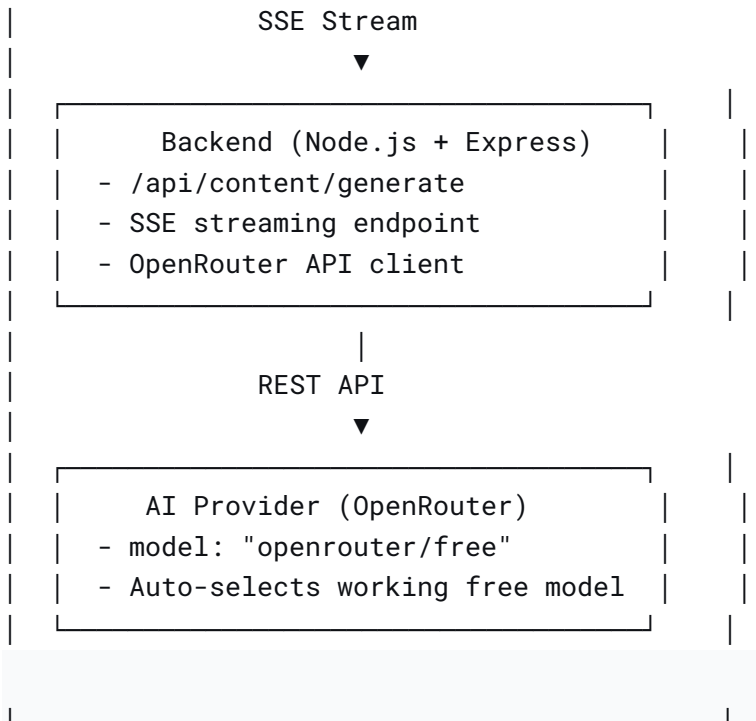
Key Learnings:

- Hardcoded API keys work for demos but should be moved to `.env`
- Using `openrouter/free` model eliminates model availability errors
- SSE streaming provides the best user experience for LLM generation

Final Stack:

text





📁 Files: `server.ts`, `README7.md`

3. Technical Implementation

Project Structure

text

```
AI_LLM_postfusion_internship/
├── Week1/
│   ├── Tuesday_task.py
│   ├── Wednesday_task.py
│   └── Thursday_task.py
├── Week2/
│   └── Few-shot prompting.py
```

```
|   |— generate_6_posts.py
|   |— constraint_test.py
|— Week3/
|   |— linkedin_generator.py
|— Week4/
|   |— generate.py
|   |— README4.md
|— Week5/
|   |— server.ts
|   |— package.json
|   |— README5.md
|— Week6/
|   |— server-simple.js
|   |— test.html
|   |— README6.md
|— Week7/
|   |— server.ts
|   |— package.json
|   |— vite.config.ts
|   |— tsconfig.json
|
|— README7.md
```

Complete `server.ts` (Week 7 Final)

typescript

```
import express from "express";
import path from "path";
import { createServer as createViteServer } from "vite";
import OpenAI from "openai";

const app = express();
const PORT = 3000;

// OpenRouter API key
const OPENROUTER_API_KEY =
  "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";

const openrouter = new OpenAI({
```

```

    baseURL: "https://openrouter.ai/api/v1",
    apiKey: OPENROUTER_API_KEY,
    defaultHeaders: {
      "HTTP-Referer": "http://localhost:3000",
      "X-Title": "Posts Fusion Studio",
    },
  });

app.use(express.json());

// Health Check
app.get("/api/health", (req, res) => {
  res.json({ status: "ok", timestamp: new Date().toISOString() });
});

// SSE Streaming Endpoint
app.post("/api/content/generate", async (req, res) => {
  const { topic = "AI Productivity", niche = "SaaS Founders", tone = "authoritative & actionable" } = req.body;

  res.setHeader("Content-Type", "text/event-stream");
  res.setHeader("Cache-Control", "no-cache");
  res.setHeader("Connection", "keep-alive");

  const sendSSE = (event: string, data: any) => {
    res.write(`event: ${event}\ndata: ${JSON.stringify(data)}\n\n`);
  };

  sendSSE("start", { topic, niche, tone });

  try {
    const prompt = `You are a world-class content creator. Write a viral, high-engagement social media post for ${niche} about "${topic}". Tone: ${tone}.\nInclude a strong hook, concise valuable insights, formatted bullet points, and a compelling call-to-action. Do not include meta-commentary.`;

    const stream = await openrouter.chat.completions.create({
      model: "openrouter/free", // Auto-selects working free model
      messages: [{ role: "user", content: prompt }],
      temperature: 0.7,
      max_tokens: 1024,
      stream: true,
    });
  }
});

```

```

    });

    for await (const chunk of stream) {
      const content = chunk.choices[0]?.delta?.content || "";
      if (content) {
        sendSSE("token", { token: content });
      }
    }
    sendSSE("done", { source: "openrouter" });
  } catch (error: any) {
    console.error("OpenRouter error:", error.message);
    sendSSE("error", { message: error.message });
    sendSSE("done", { source: "error" });
  }

  res.end();
});

// Start server
async function startServer() {
  if (process.env.NODE_ENV !== "production") {
    const vite = await createViteServer({
      server: { middlewareMode: true },
      appType: "spa",
    });
    app.use(vite.middlewares);
  }

  app.listen(PORT, "0.0.0.0", () => {
    console.log(`✅ Server running on http://localhost:${PORT}`);
    console.log(`🔑 OpenRouter API key loaded`);
  });
}

startServer();

```

4. Prompt Engineering

My Evolution in Prompting

Before (Week 1)	After (Week 7)
Writing one good prompt	Writing, testing, and iterating on prompts
One-time instruction	Continuous refinement process
"AI will figure it out"	"I must explicitly tell it everything"
Generic outputs	Brand-specific, personalized outputs
No validation	Validation and re-prompting

Best Practices Discovered

1. Always use few-shot examples – The AI learns from examples better than instructions
2. Repeat constraints 3 times – "150-300 words. Strictly between 150 and 300 words. Word count: 150-300 words."
3. Validate outputs – Always check word count, hashtags, structure
4. Iterate at least 2-3 times – First attempt is almost never perfect
5. Specify tone explicitly – "Bold, direct, opinionated" vs just "professional"

Example: LinkedIn Post Generator Prompt

```
python
```

```
system_prompt = f"""  
ROLE: You are a professional content creator and industry expert in the  
"{niche}" niche.  
Your task is to write high-quality LinkedIn posts with a "{tone}" tone.
```

AUDIENCE: Working professionals and peers in the {niche} industry.

STRUCTURE RULES:

- 1. HOOK: Start with a relatable problem or surprising fact.
- 2. BODY: Provide exactly 3-4 actionable insights.
- 3. CALL-TO-ACTION: End with a question to encourage comments.

LENGTH: Between 200-300 words.

""

5. API Integration

APIs Used & Comparison

API	Speed	Cost	Best For
Ollama	Medium	Free	Local testing, offline use
Groq	Very Fast (100-300ms)	Free tier	Production, speed-critical apps
OpenRouter	Fast (varies)	Free tier	Access to 50+ models, auto-fallback
Gemini	Medium	Free tier	Multimodal tasks, long context

OpenRouter Models Tested

Model Slug	Status	Best For
<code>openrouter/free</code>	✅ Working	Auto-selects working free model
<code>meta-llama/llama-3.1-8b-ins</code> <code>truct:free</code>	❌ Unavailable	(Paid version available)
<code>microsoft/phi-3.5-mini-128k</code> <code>-instruct:free</code>	❌ Unavailable	(No longer free)
<code>google/gemini-2.0-flash-lit</code> <code>e</code>	❌ Unavailable	(No longer free)

Lesson Learned: Free models on OpenRouter change frequently. Use `openrouter/free` to avoid constant updates.

6. Full Stack Development

React + Node.js Streaming App

Frontend (React + Tailwind):

```
tsx
```

```

function App() {
  const [content, setContent] = useState('');
  const [isGenerating, setIsGenerating] = useState(false);

  const generatePost = async () => {
    setIsGenerating(true);
    const response = await fetch('/api/content/generate', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ topic: 'AI in Healthcare' })
    });

    const reader = response.body.getReader();
    const decoder = new TextDecoder();

    while (true) {
      const { value, done } = await reader.read();
      if (done) break;
      const chunk = decoder.decode(value);
      // Parse SSE events and update UI
      // ...
    }
    setIsGenerating(false);
  };

  return (
    <div className="p-8">
      <button onClick={generatePost}>Generate Post</button>
      <pre>{content}</pre>
    </div>
  );
}

```

Backend (Express + SSE):

typescript

```

app.post('/api/content/generate', async (req, res) => {
  res.setHeader('Content-Type', 'text/event-stream');
  res.setHeader('Cache-Control', 'no-cache');

```

```
const sendSSE = (event, data) => {
  res.write(`event: ${event}\ndata: ${JSON.stringify(data)}\n\n`);
};

const stream = await openrouter.chat.completions.create({
  model: "openrouter/free",
  messages: [{ role: 'user', content: prompt }],
  stream: true,
});

for await (const chunk of stream) {
  const content = chunk.choices[0]?.delta?.content || '';
  if (content) {
    sendSSE('token', { token: content });
  }
}

sendSSE('done', { source: 'openrouter' });
res.end();

});
```

7. Sample Outputs

Output 1 – Authoritative Tone (Groq)

text

AI in Healthcare: The \$60B Opportunity Founders Can't Ignore

The healthcare industry is ripe for disruption—and AI is the catalyst. By 2025, the AI healthcare market will hit \$60 billion...

Here's why you must act now:

- Accelerate Drug Discovery: AI slashes R&D timelines by 50%+
- Personalized Medicine: Algorithms analyze genomics + wearables
- Operational Efficiency: Reduce admin costs with AI-driven workflows

#AIInHealthcare #SaaS #HealthTech #Innovation #FutureOfMedicine

Validation:

- Word Count: 247 
 - Hashtags: 5/5 
 - Valid Status: PASSED 
-

Output 2 – Educational Tone (OpenRouter)

text

AI in Healthcare: A \$188 Billion Opportunity for SaaS Founders

Hook: Imagine a world where AI diagnoses diseases faster than humans...

Why AI in Healthcare?

- Massive Demand: 72% of hospitals plan to adopt AI by 2025
- High Margins: AI-driven SaaS solutions scale globally
- Regulatory Tailwinds: FDA's AI/ML action plan accelerates approvals

#AI #HealthTech #SaaS #HealthcareInnovation #StartupGrowth

Output 3 – Streaming Experience (React UI)

As the server generates content, it streams token by token:

text

AI → AI in → AI in Healthcare → AI in Healthcare: → AI in Healthcare: A →

AI in Healthcare: A \$188 → AI in Healthcare: A \$188 Billion → ... (complete post)

8. Challenges & Solutions

Challenge	Solution
Ollama model errors	Switched to Groq API
Groq model became paid	Switched to OpenRouter with <code>openrouter/free</code>
<code>.env</code> variables not loading	Hardcoded API key temporarily
CORS blocking frontend requests	Served frontend from same server
Memory issues (esbuild)	Increased Node memory: <code>NODE_OPTIONS="--max-old-space-size=4096"</code>
Model 404 errors	Used <code>openrouter/free</code> to auto-select working model

React UI not connecting	Served index.html from Express server
-------------------------	---------------------------------------

9. Skills Acquired

Skill	Level
Python Programming	Intermediate
API Integration	Intermediate
Prompt Engineering	Proficient
LinkedIn Content Generation	Proficient
Tone-Specific Prompting	Proficient
React + TypeScript	Intermediate
Node.js + Express	Intermediate
GitHub	Proficient
Docker	Basic

10. Conclusion

What I Learned

1. Prompt engineering is iterative – You cannot write a perfect prompt on the first try. You must generate, validate, tweak, and regenerate.
2. Model selection matters – Different models produce different quality outputs. Choose the right model for the task.
3. Streaming improves UX – Users prefer seeing content appear word-by-word rather than waiting for the full response.
4. Free tiers change – APIs frequently update their free offerings. Always have a fallback strategy.
5. Documentation is key – Good documentation helps you and others understand, maintain, and extend your project.

Key Accomplishments

- ✓ Built a LinkedIn post generator using multiple APIs (Ollama, Groq, OpenRouter)
- ✓ Implemented real-time streaming with Server-Sent Events
- ✓ Created a full-stack React + Node.js application
- ✓ Learned prompt engineering best practices through iteration
- ✓ Documented the entire internship journey

Future Improvements

- Migrate to `google.genai` package for Gemini support
 - Add user authentication and saved posts
 - Implement model comparison feature
 - Deploy to cloud (Vercel/Railway)
 - Add analytics and usage tracking
-

Submission Files

Required Documentation:

1. `Internship_Completion_Report.md` (this file)
2. `README.md` (for each week)
3. `server.ts` (Week 7)
4. `package.json` (Week 7)
5. Sample outputs (images/screenshots)

Optional:

- Week 1-7 code files
 - `.env.example`
 - `metadata.json`
-

Quick Links

- [GitHub Repository](#)
 - [OpenRouter API](#)
 - [Groq Console](#)
 - [Ollama](#)
-

 Prepared by: Sawaira Mumtaz

 Date: August 17, 2026

 Company: Posts Fusion / Pro Fusion AI

Thank you for the opportunity to complete this internship! 🚀